

COMP4601 Project Report: Audio Fingerprinting Acceleration

Team: Esha Tripathi, Venus Baranwal, Rukhsaar Aujla, Netik Kumar Maheshwari

1. Summary & System Baseline

This project accelerates the core component of a C++ port of the Dejavu audio fingerprinting algorithm. The system processes 22.05 kHz PCM audio into 4096-sample windows with a 50% overlap (HOP = 2048), applies Hann windowing, and computes a 1D FFT per frame to generate a log-power magnitude spectrogram. The core of the fingerprinting logic requires finding local maxima using a 41x41 diamond mask, followed by a threshold filter (Amplitude > 10) and an anchor peak targeting mechanism (Fan-out = 15) to generate hash signatures.

Initial profiling of the software-only implementation on the ARM processor (PS) yielded a total execution time of 16,692 ms. The breakdown revealed a severe computational hotspot:

- Windowing: 14.86 ms
- FFT + Spectrogram: 392.447 ms
- Peak Detection: 16,284.7 ms (97.45% of total runtime)
- Hashing and JSON: 7.855 ms

Guided by Amdahl's Law, the peak detection stage was isolated PL acceleration, yielding a final kernel speedup of 4071x (from 16.28s to 4.00 ms) and an end-to-end system **speedup of 16.5** times (approx. **41.25% of the theoretical max speedup**(40.25%)).

2. Hardware Architecture & Design Evolution

The peak detection kernel evolved through seven iterations, with each version addressing a specific hardware bottleneck. The main challenges were **memory bandwidth, compute-resource utilisation, and AXI memory stalls**. The final V7 architecture combines line buffering, register partitioning, compile-time masking, binary reduction, and local output staging to achieve an II=1 pipeline.

2.1. Memory Bandwidth Problem

Problem (V1-V3)

The baseline implementation accessed the 41x41 neighbourhood directly from external

DDR memory. This created excessive memory traffic and prevented the sliding window from being updated every cycle. Storing the window in BRAM (2 ports) also limited the number of simultaneous accesses, resulting in an II=3 pipeline.

Solution

A line buffer was introduced to retain the rows required by the sliding window. As each new spectrogram column arrives, the line buffer provides the corresponding 41 values needed to update the window, greatly reducing dependence on DDR accesses.

The 41x41 window was then completely partitioned using *#pragma HLS ARRAY_PARTITION complete*.

This maps the 1681 window elements to independently accessible registers rather than BRAM.

The resulting architecture uses **pipelining and unrolling together**. The `shift_window` loop is pipelined at II=1, allowing a new window position to be processed every clock cycle, while the inner window-shift and column-loading loops are fully unrolled so that the required register updates occur in parallel.

Result

The entire window can be shifted in parallel in one clock cycle, providing the data bandwidth required for an II=1 pipeline.

2.2. Compute Logic Problem

Problem (V2 & V5)

For each window position, peak detection evaluates the maximum value within a diamond-shaped neighbourhood of radius 20. The neighbourhood contains 841 valid positions. Naively unrolling these comparisons created excessive parallel logic, reaching **169.15%** LUT utilisation in V2. V5 reduced parallelism by relaxing the pipeline to II=6, but LUT utilisation remained above the available resources at **149.0%**.

Solution (V7)

V7 combined compile-time masking with a balanced binary reduction tree.

First, the diamond boundary is represented by a compile-time *constexpr DiamondMask*. This removes the need to calculate the Manhattan-distance condition at runtime and allows invalid positions to be eliminated during synthesis.

The remaining 841 values are then processed using a balanced reduction tree. The values are padded to 1024 elements, allowing a power-of-two tree with 10 reduction stages:

512 -> 256 -> 128 -> ... -> 2 -> 1

Each stage is fully unrolled, creating the required parallel comparators. The tree therefore trades additional LUT usage for throughput, allowing the maximum to be reduced through a fixed-depth hierarchy rather than sequentially scanning all 841 values.

Result

The reduction tree supports the required II=1 pipeline while reducing LUT utilisation to **87.0%**, making the design feasible on the board.

2.3. AXI Bus Stalls

Problem (V1-V5)

Detected peaks were initially written directly to external DDR from inside the peak-detection loop. These irregular writes prevented the HLS compiler from generating efficient sequential Axi accesses, introducing memory stalls and limiting burst-write opportunities.

Solution (V6-V7)

Peak coordinates were instead stored in local BRAM buffers during computation. A separate sequential write_axi loop transfers the completed results to DDR after peak detection finishes.

Result:

Peak detection is no longer blocked by irregular DDR writes. The final transfer can be performed as a sequential burst, reducing Axi overhead to **0.0226 ms**.

2.4. Data Type Optimisation

From V3 onwards, the hardware datapath was converted from 32-bit floating-point values to ap_fixed<16,10>.

This reduces the data width from 32 to 16 bits, reducing memory requirements and matching the 16-bit hardware data path more naturally. Fixed-point arithmetic also avoids the need for floating-point hardware for the peak-detection operations.

However, this optimisation introduced a CPU-side performance trade-off as using ap_fixed for the spectrogram data also slowed the PS FFT. This issue is analysed separately in Section 4.

The evolution of our design and the comparison metrics we used (throughput, latency, resource utilisation) can be seen in table 1.

Version	Optimisation	LUT (%)	FF (%)	BRAM (%)	II / Latency Issue	Total Latency (ms)	Speedup Factor (x)	Throughput (Windows/s)
V1 (Unaccelerated)	Baseline HLS code; II=3 memory dependency on find_max.	6.28% (7.3k)	1.83% (4.2k)	7.29%	Latency: 25.3 μ s per loop	5,005.00	1x	0.2
V2 (Full Unroll)	Naive #pragma HLS UNROLL generated 841 raw comparators without tree reduction.	169.15%	138.02%	14.93%	Over-utilised logic	21.287	235.1x	46.98
V3 (Bit-width Reduction)	Switched to ap_fixed<32,16> & cyclic partition; memory bottleneck persisted.	85.00% (100k)	94.00% (221k)	5.00%	II Memory Dependency	50	100.1x	20
V4 (Partial unrolling)	Added partial partitioning and unrolling instead to save resources but still exceeded the limits	171% (200k)	123.00% (288k)	15.00%	Over-budget	18.5	270.5x	54.05
V5 (Static Mask)	Added compile-time DiamondMask; pruned invalid gates but still exceeded logic.	149.00% (174k)	105.00% (245k)	14.00%	Over-budget	14.35	348.8x	69.69
V6 (AXI Burst Fix)	Fixed AXI burst transfers; relaxed II to fit within resource limits.	79.00% (92.5k)	60.00% (140k)	24.00%	Fits device (19.9 ms)	19.9	251.5x	50.25
V7 (Golden Version)	Balanced 1024-node Tree Reduction + Local BRAM peak staging.	87.00% (101k)	67.00% (156k)	24.00%	Pipelined II=1 (4.00 ms)	3.514	1424.3x	284.58

Table 1: Comprehensive progression & performance comparison detailing optimisation strategy, resource utilisation, execution latency, speedup, and processing throughput

3. Analysis & Performance Evaluation

The **final profiling results** were as follows:

- FFT + Spectrogram: 983.596 ms
- Windowing: 14.8365 ms
- Peak Detection (XRT Transfer & Kernel Exec): 4.00054 ms
- Hashing and JSON: 7.80151 ms
- Transfer overhead: 0.022631 ms

3.1. Final Speedup & Transfer Overheads

The baseline peak detection execution time on the ARM PS was 16,284.90 ms. By shifting this to the kernel, the Golden Version (V7) completed the task in 4.00 ms, achieving a final 4,071x kernel speedup.

The 4.00 ms includes only **0.0226 ms of AXI transfer overhead**. V7 therefore keeps data-transfer costs negligible by staging peak outputs in local BRAM and performing a single burst write.

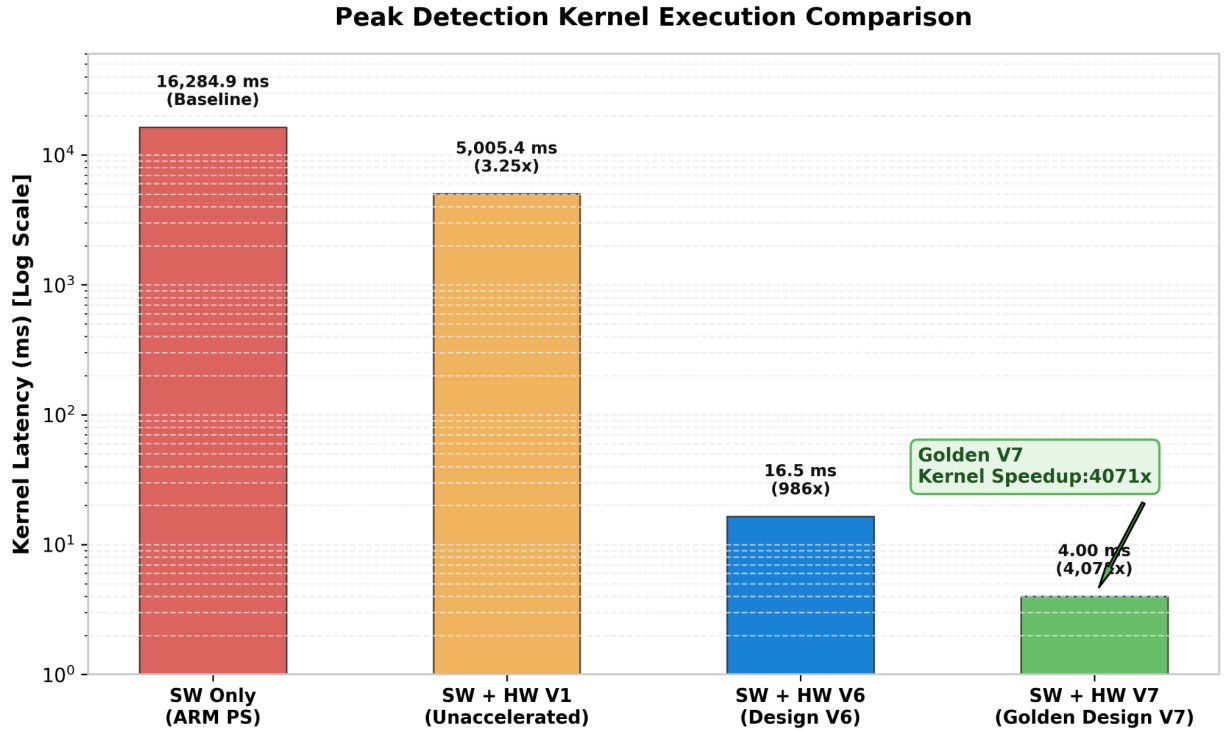


Figure 1: Execution time of the detect_peaks kernel across implementations. Software execution on the ARM PS required 16,284.90 ms. The unaccelerated HW+SW baseline (V1) reduced latency to 5,005.44 ms (3.25x speedup), while the V6 reached 16.51 ms (986x speedup). The final golden design (V7) achieved a kernel execution time of **4.00 ms**, representing a **4,071x speedup** over pure software execution.

3.2. Throughput Comparison: V6 vs V7

To demonstrate the architectural efficiency of V7, we compare it against our first mathematically feasible design (V6). Task throughput is calculated as:

$$\text{Throughput} = 1 / \text{Execution Time (s)}$$

- Design V6: To fit the board's resource limits without a reduction tree, the pipeline was explicitly relaxed to $II=6$.
 - Execution Time: **16.51 ms**
 - Throughput: $1 / 0.01651 \text{ s} \approx \mathbf{60.57 \text{ executions/second}}$
- Golden Design V7 introduced the 1024-node balanced binary reduction tree allowing the pipeline to achieve an ideal $II=1$.
 - Execution Time: **4.00 ms**
 - Throughput: $1 / 0.00400 \text{ s} = \mathbf{250 \text{ executions/second}}$

Thus, V7 increases throughput by approximately **4.1x** over V6 while remaining within the board's resource constraints. V7 was therefore selected as the final design.

3.3. LUT and FF analysis

The Golden Version utilises 87.0% of available LUTs and 67.0% of available Flip-Flops (FFs). These utilisation numbers are driven by the window storage and reduction tree.

High FF (67.0%) utilisation comes from 2D Window Registers. Applying `#pragma HLS ARRAY_PARTITION complete` to the 41x41 window forced the synthesiser to map all 1,681 fixed-point elements into discrete FFs. This massive register allocation is intentional as it allows the entire 2D grid to shift left simultaneously in 1 clock cycle using pure physical wire routing.

High LUT (87.0%) utilisation comes from the 1024-Node Reduction Tree. To evaluate 841 points in a single cycle ($II=1$), the unrolled `find_diamond_max` loop generates hundreds of parallel 16-bit comparators. Because DSP slices are meant for multiplication, these logical comparators must be fabricated from LUTs. However, by generating a `constexpr DiamondMask`, we forced the compiler to prune invalid points at compile-time. This kept LUT usage safely below 100%, whereas earlier versions without the mask blew up to an infeasible 169.15%.

3.4. FFT Performance

While peak detection latency decreased drastically, the FFT generation execution time paradoxically jumped from ~392.4 ms to 983.60 ms.

This is because we used `ap_fixed<16,10>` for the CPU-side spectrogram data. While `ap_fixed` is efficient for FPGA arithmetic, the ARM processor has no dedicated fixed-point hardware. Its overloaded operators therefore introduce software-based shifting and masking for each arithmetic operation.

The solution is to decouple CPU and FPGA data types. We can perform FFT computation using standard 32-bit floating point (`CPU_DTYPE`) on the ARM CPU, then cast to `ap_fixed<16,10>` (`HW_DTYPE`) when transferring the spectrogram to the FPGA. This conversion might add a slight transfer overhead but that's acceptable given the massive reduction in FFT compute time.

The overall speed-up and comparison can be seen in Figure 2. Needless to say, each design's correctness was verified by a testbench and multiple audio files.

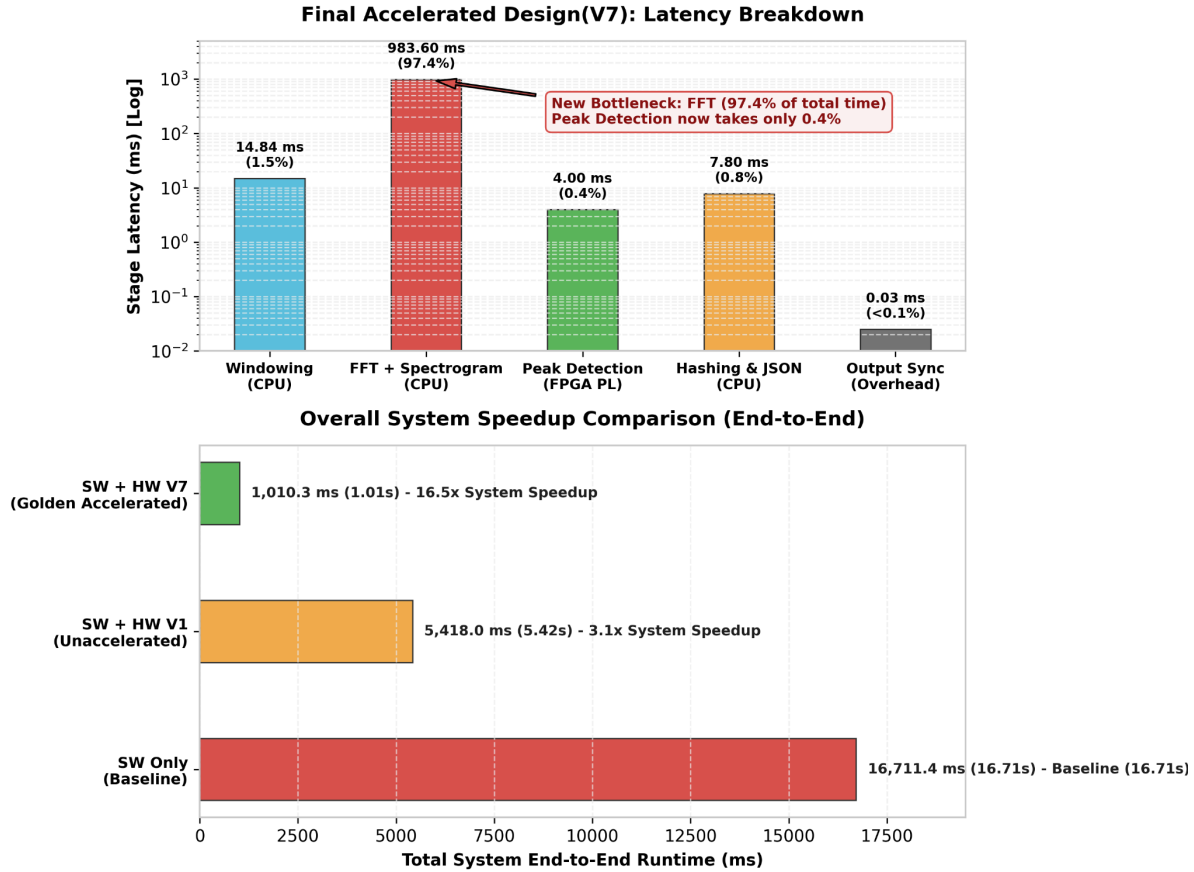


Figure 2: End-to-end application profiling and speedup for the V7 pipeline. (a) Per-stage latency breakdown showing peak detection accelerated to 4.00 ms (0.4% of total time), shifting the primary system bottleneck to CPU-bound FFT/spectrogram processing (983.60 ms, 97.4%). (b) End-to-end runtime comparison demonstrating total execution time reduction from 16.71s (software-only) to 1.01s, delivering a 16.5x overall system speedup.

4. Future Work

With peak detection successfully accelerated to 4.00 ms, the new bottleneck is the `compute_spectrogram` function. FFT and Spectrogram generation now account for 97.4% (983.60 ms) of total execution time. Future optimisations must target this new bottleneck across both the software interface and hardware logic.

4.1. Rectifying CPU Data Type Deceleration

The current architecture applies the `ap_fixed<16, 10>` data type across the entire pipeline to satisfy hardware constraints. However, because the ARM PS lacks a dedicated fixed-point arithmetic unit, executing the FFT in software using C++ overloaded fixed-point operators caused the CPU preprocessing time to balloon to 983.60 ms. To restore the CPU's original performance, the data types must be decoupled:

- We can use floats in the preprocessing stage on the PS to restore the original CPU preprocessing execution time of approximately 392 ms and maintain `ap_fixed<16, 10>` for the FPGA kernel to preserve DSP and LUT efficiency.
- We can cast data from float to `ap_fixed` dynamically during the xRT buffer mapping. This introduces an additional $O(N)$ conversion overhead but avoids the ~590 ms fixed-point CPU penalty.

4.2. Kernel Fusion: Accelerating the FFT

To push the system toward its theoretical maximum speedup, the next architectural step is **Kernel Fusion**: moving the FFT calculation onto the Programmable Logic (PL) and fusing it with the peak detection kernel.

Based on the DFT from lab 2, hardware acceleration of the FFT is highly feasible.

Using `ap_fixed<40, 16>` precision, coefficient tables, an unroll factor of 8, and an array cyclic partition factor of 8 (providing 16 memory ports), the hardware DFT achieved an Initiation Interval (II) of 1. Execution time was 229.18 μ s at 200 MHz.

An $O(N^2)$ DFT for $N = 256$ requires $256^2 = 65,536$ Multiply-Accumulate (MAC) operations. At 229.18 μ s (45,836 cycles), the hardware achieved a throughput of roughly 0.7 cycles per MAC via pipeline unrolling.

We use a 4096-point Radix-2 FFT, requiring approximately 24,576 butterfly operations per window compared with 65,536 operations for the 256-point DFT. As a rough performance projection, assuming the FFT achieves a similar computational efficiency to the Lab 02 DFT gives approximately 17,203 cycles (86 μ s) per window, or **8.60 ms for 100 windows**.

4.3. Resource Mitigation Strategies

The primary challenge of kernel fusion is resource availability. The Golden peak detection kernel (V7) currently utilises 87.00% of available LUTs. Adding an FFT core could cause LUT exhaustion. To account for this, the following structural refinements will be implemented:

4.3.1. Dataflow Pipelining (`#pragma HLS DATAFLOW`)

Currently, the FPGA pipeline executes sequentially: it waits 101 cycles to read `new_row` from DDR, then spends 121 cycles processing `shift_window`. Applying `#pragma HLS DATAFLOW` with row buffers would allow the DDR AxI reads to occur concurrently with the window shifting, potentially dropping the iteration latency from ~306 cycles to ~125 cycles, allowing us to safely reduce the spatial unroll factor if necessary to save LUTs without sacrificing throughput.

4.3.2. Bit-width Pruning

We can change `local_peak_freq` from `uint16_t` to `ap_uint<12>` (sufficient for `MAX_FREQ = 2049`) to save (approx 25%) BRAM storage and `local_peak_time` to `ap_uint<7>` (sufficient for `MAX_WINDOWS = 100`) to save (about 56%) BRAM storage.

4.3.3. DSP Utilisation

The FFT is expected to primarily use DSP resources, while peak detection is LUT-heavy, allowing the two kernels to potentially coexist with limited resource contention.

4.4. Estimated Improvement

Assuming the projected FFT performance and a fully streaming dataflow architecture:

- **Projected FFT interval = 8.60 ms**
- **Peak Detection = 3.51 ms**
- **Projected steady-state pipeline interval $\approx$ 8.60 ms/spectrogram**

The stages can overlap because peak detection can consume FFT output as it is produced. The actual end-to-end latency will depend on pipeline fill/drain and transfer overhead. Using the projected 8.60 ms kernel interval:

Estimated End-to-End Runtime = 14.86 ms (Windowing) + 8.60 ms (Fused Kernel) + 7.85 ms (Hashing) $\approx$ 31.31 ms

This corresponds to a projected end-to-end speedup of approximately **533x** over the 16,692 ms baseline.